

Programació recursiva metòdica (II)

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

- 1 Immersió o generalització d'una funció
 - Immersió per afebliment de la postcondició
 - Immersió per reforçament de la precondició
 - Transformació d'alg. recursius lineals finals a alg. iteratius
- 2 Exercicis

Índex

1

Immersió o generalització d'una funció

- Immersió per afebliment de la postcondició
- Immersió per reforçament de la precondició
- Transformació d'alg. recursius lineals finals a alg. iteratius

2

Exercicis

Immersió o generalització d'una funció

- Per què?

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.
 - Afegint paràmetres i resultats addicionals.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.
 - Afegint paràmetres i resultats addicionals.
- Si modifiquem els paràmetres i/o resultats $\Rightarrow$ haurem de canviar l'especificació de la funció.

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.
 - Afegint paràmetres i resultats addicionals.
- Si modifiquem els paràmetres i/o resultats $\Rightarrow$ haurem de canviar l'especificació de la funció.
- Com recuperem la funció inicial?

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.
 - Afegint paràmetres i resultats addicionals.
- Si modifiquem els paràmetres i/o resultats $\Rightarrow$ haurem de canviar l'especificació de la funció.
- Com recuperem la funció inicial?
 - Fixant els paràmetres addicionals (donant-los un valor fix).

Immersió o generalització d'una funció

- Per què?
 - Per poder fer un programa recursiu.
 - Per transformar un programa recursiu en un altre més senzill o eficient.
- Com es generalitza una funció?
 - Afegint paràmetres addicionals.
 - Afegint resultats addicionals.
 - Afegint paràmetres i resultats addicionals.
- Si modifiquem els paràmetres i/o resultats $\Rightarrow$ haurem de canviar l'especificació de la funció.
- Com recuperem la funció inicial?
 - Fixant els paràmetres addicionals (donant-los un valor fix).
 - Rebutjant els resultats addicionals.

Immersiones d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:

Immersions d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:
 - ① Afeblir la postcondició (la nova postcondició és més feble, més fàcil de satisfer).

Immersiones d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:
 - 1 Afeblir la postcondició (la nova postcondició és més feble, més fàcil de satisfer).
 - 2 Reforçar la precondició (la nova precondició és més forta, té més condicions).

Immersiones d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:
 - 1 Afeblir la postcondició (la nova postcondició és més feble, més fàcil de satisfer).
 - 2 Reforçar la precondició (la nova precondició és més forta, té més condicions).
- Segons si fem (1) o (2), el tipus d'immersió s'anomena:

Immersiones d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:
 - ① Afeblir la postcondició (la nova postcondició és més feble, més fàcil de satisfer).
 - ② Reforçar la precondició (la nova precondició és més forta, té més condicions).
- Segons si fem (1) o (2), el tipus d'immersió s'anomena:
 - ① Immersió d'especificació per **afebliment de la postcondició**.

Immersions d'especificació

- Hem dit que canviar els paràmetres i/o resultats $\Rightarrow$ canviar l'especificació de la funció. Tècniques:
 - 1 Afeblir la postcondició (la nova postcondició és més feble, més fàcil de satisfer).
 - 2 Reforçar la precondició (la nova precondició és més forta, té més condicions).
- Segons si fem (1) o (2), el tipus d'immersió s'anomena:
 - 1 Immersió d'especificació per **afebliment de la postcondició**.
 - 2 Immersió d'especificació per **reforçament de la precondició**.

Índex

1 Immersió o generalització d'una funció

- Immersió per afebliment de la postcondició
- Immersió per reforçament de la precondició
- Transformació d'alg. recursius lineals finals a alg. iteratius

2 Exercicis

Immersió per afebliment de la postcondició

- Per què?

Immersió per afebliment de la postcondició

- Per què? Perquè la funció original no admet una solució recursiva.

Immersió per afebliment de la postcondició

- Per què? Perquè la funció original no admet una solució recursiva.
- Com és la funció recursiva respecte la funció original?

Immersió per afebliment de la postcondició

- Per què? Perquè la funció original no admet una solució recursiva.
- Com és la funció recursiva respecte la funció original?
 - Té una postcondició més feble que la funció original.

Immersió per afebliment de la postcondició

- Per què? Perquè la funció original no admet una solució recursiva.
- Com és la funció recursiva respecte la funció original?
 - Té una postcondició més feble que la funció original.
 - Té més paràmetres que la funció original.

Exemple 1: immersió per afebliment de la `Post` (I)

Dissenyar una funció que donat un vector no buit d'enters, retorni la suma de tots els seus elements.

Exemple 1: immersió per afebliment de la `Post` (I)

Dissenyar una funció que donat un vector no buit d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els elements
        del vector v */
```

Exemple 1: immersió per afebliment de la `Post` (I)

Dissenyar una funció que donat un vector no buit d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els elements
        del vector v */
```

- Per poder obtenir un disseny recursiu necessitem garantir que l'execució acabarà (funció de fita) $\Rightarrow$ algun paràmetre decreix a cada crida recursiva.

Exemple 1: immersió per afebliment de la `Post` (I)

Dissenyar una funció que donat un vector no buit d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els elements
        del vector v */
```

- Per poder obtenir un disseny recursiu necessitem garantir que l'execució acabarà (funció de fita) $\Rightarrow$ algun paràmetre decreix a cada crida recursiva.
- La funció `suma_vect_int` no té cap paràmetre que compleixi aquesta condició.

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: El resultat és la suma de v[0..i] */
```

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: El resultat és la suma de v[0..i] */
```

- A la `Post` hi apareix una variable, `i`, que passa a ser un **nou paràmetre** de la funció.

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: El resultat és la suma de v[0..i] */
```

- A la `Post` hi apareix una variable, `i`, que passa a ser un **nou paràmetre** de la funció.
- A la `Pre` hem d'afegir les condicions de `i`.

Exemple 1: immersió per afebliment de la `Post` (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: El resultat és la suma de v[0..i] */
```

- A la `Post` hi apareix una variable, `i`, que passa a ser un **nou paràmetre** de la funció.
- A la `Pre` hem d'afegir les condicions de `i`.
- Especificació de la funció recursiva:

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
```

Exemple 1: immersió per afebliment de la `Post` (III)

Implementació de la funció recursiva i justificació:

Exemple 1: immersió per afebliment de la `Post` (III)

Implementació de la funció recursiva i justificació:

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
{
    int suma;
    if (i == 0) suma = v[0];
    else {
        suma = i_suma_vect_int(v, i-1);
        /* HI: "suma" és la suma de v[0..i-1] */
        suma = suma + v[i];
    }
    return suma;
}
```

Exemple 1: immersió per afebliment de la `Post` (III)

Implementació de la funció recursiva i justificació:

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
{
    int suma;
    if (i == 0) suma = v[0];
    else {
        suma = i_suma_vect_int(v, i-1);
        /* HI: "suma" és la suma de v[0..i-1] */
        suma = suma + v[i];
    }
    return suma;
}
```

- Cas senzill: Si $i=0$, la postcondició indica que el resultat ha de ser $v[0..0]$, per tant assignem a `suma` el valor $v[0]$.

Exemple 1: immersió per afebliment de la `Post` (III)

Implementació de la funció recursiva i justificació:

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
{
    int suma;
    if (i == 0) suma = v[0];
    else {
        suma = i_suma_vect_int(v, i-1);
        /* HI: "suma" és la suma de v[0..i-1] */
        suma = suma + v[i];
    }
    return suma;
}
```

- Cas senzill: Si $i=0$, la postcondició indica que el resultat ha de ser $v[0..0]$, per tant assignem a `suma` el valor $v[0]$.
- Cas recursiu: Si $i \neq 0$, el resultat és $v[0..i-1] + v[i]$. **HI**.

Exemple 1: immersió per afebliment de la `Post` (III)

Implementació de la funció recursiva i justificació:

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
{
    int suma;
    if (i == 0) suma = v[0];
    else {
        suma = i_suma_vect_int(v, i-1);
        /* HI: "suma" és la suma de v[0..i-1] */
        suma = suma + v[i];
    }
    return suma;
}
```

- Cas senzill: Si $i=0$, la postcondició indica que el resultat ha de ser $v[0..0]$, per tant assignem a `suma` el valor $v[0]$.
- Cas recursiu: Si $i \neq 0$, el resultat és $v[0..i-1] + v[i]$. **HI**.
- Acabament: i decreix a cada crida recursiva.

Exemple 1: immersió per afebliment de la `Post` (IV)

Implementació de la funció original:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els
    elements del vector v */
{
    return i_suma_vect_int(v, ?);
}
```

Exemple 1: immersió per afebliment de la `Post` (V)

Implementació de la funció original:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els
    elements del vector v */
{
    return i_suma_vect_int(v, v.size() - 1);
}
```

Exercici: divisors naturals d' n en una pila

Donat un enter n més gran que zero, s'han d'obtenir tots els seus divisors naturals **menors** que ell en una pila ordenada en ordre decreixent des del cim. Per exemple, si $n = 24$, la pila ha de ser:

12
8
6
4
3
2
1

Solució: divisors naturals d' n en una pila (I)

Especificació:

```
void divisors(int n, stack<int> &p)
/* Pre: n>0, p és buida */
/* Post: p conté els divisors d'n menors que ell ,
        en ordre decreixent des del cim */
```

Solució: divisors naturals d' n en una pila (I)

Especificació:

```
void divisors(int n, stack<int> &p)
/* Pre: n>0, p és buida */
/* Post: p conté els divisors d'n menors que ell ,
    en ordre decreixent des del cim */
```

- Per poder obtenir un disseny recursiu necessitem garantir que algun paràmetre decreix a cada crida recursiva.

Solució: divisors naturals d' n en una pila (I)

Especificació:

```
void divisors(int n, stack<int> &p)
/* Pre: n>0, p és buida */
/* Post: p conté els divisors d'n menors que ell ,
    en ordre decreixent des del cim */
```

- Per poder obtenir un disseny recursiu necessitem garantir que algun paràmetre decreix a cada crida recursiva.
- La funció `divisors` no té cap paràmetre que compleixi aquesta condició.

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: p conté els divisors d'n menors que m,  
en ordre decreixent des del cim */
```

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: p conté els divisors d'n menors que m,  
en ordre decreixent des del cim */
```

- A la `Post` hi apareix una variable, `m`, que passa a ser un **nou paràmetre** de la funció.

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: p conté els divisors d'n menors que m,  
   en ordre decreixent des del cim */
```

- A la `Post` hi apareix una variable, `m`, que passa a ser un **nou paràmetre** de la funció.
- A la `Pre` hem d'afegir les condicions de `m`.

Solució: divisors naturals d' n en una pila (II)

- Afeblim la postcondició fent que el resultat de la funció recursiva calculi “menys coses” que la funció original.
- Proposta:

```
/* Post: p conté els divisors d'n menors que m,
   en ordre decreixent des del cim */
```

- A la `Post` hi apareix una variable, `m`, que passa a ser un **nou paràmetre** de la funció.
- A la `Pre` hem d'afegir les condicions de `m`.
- Especificació de la funció recursiva:

```
void i_divisors(int n, stack<int> &p, int m)
/* Pre: n >= m > 0, p és buida */
/* Post: p conté els divisors d'n menors que m,
   en ordre decreixent des del cim */
```

Solució: divisors naturals d' n en una pila (III)

Implementació de la funció original:

Solució: divisors naturals d' n en una pila (III)

Implementació de la funció original:

```
void divisors(int n, stack<int> &p)
/* Pre: n>0, p és buida */
/* Post: p conté els divisors d'n menors que ell ,
    en ordre decreixent des del cim */
{
    i_divisors(n, p, n);
}
```

Solució: divisors naturals d' n en una pila (IV)

Implementació de la funció recursiva i justificació:

Solució: divisors naturals d' n en una pila (IV)

Implementació de la funció recursiva i justificació:

```
void i_divisors(int n, stack<int> &p, int m)
/* Pre: n >= m > 0, p és buida */
/* Post: p conté els divisors d'n menors que m,
en ordre decreixent des del cim */
{
    if (m > 1) {
        i_divisors(n, p, m-1);
        /* HI: p conté els divisors d'n menors que m-1,
en ordre decreixent des del cim */
        if (n % (m-1) == 0) p.push(m-1);
    }
}
```

Solució: divisors naturals d' n en una pila (IV)

Implementació de la funció recursiva i justificació:

```
void i_divisors(int n, stack<int> &p, int m)
/* Pre:  $n \geq m > 0$ , p és buida */
/* Post: p conté els divisors d' $n$  menors que m,
en ordre decreixent des del cim */
{
    if (m > 1) {
        i_divisors(n, p, m-1);
        /* HI: p conté els divisors d' $n$  menors que m-1,
en ordre decreixent des del cim */
        if (n % (m-1) == 0) p.push(m-1);
    }
}
```

- Cas senzill: Si $m=1$, no hi ha cap divisor d' n menor que m . No afegim res a p .

Solució: divisors naturals d' n en una pila (IV)

Implementació de la funció recursiva i justificació:

```
void i_divisors(int n, stack<int> &p, int m)
/* Pre:  $n \geq m > 0$ ,  $p$  és buida */
/* Post:  $p$  conté els divisors d' $n$  menors que  $m$ ,
en ordre decreixent des del cim */
{
    if (m > 1) {
        i_divisors(n, p, m-1);
        /* HI:  $p$  conté els divisors d' $n$  menors que  $m-1$ ,
en ordre decreixent des del cim */
        if (n % (m-1) == 0) p.push(m-1);
    }
}
```

- Cas senzill: Si $m=1$, no hi ha cap divisor d' n menor que m . No afegim res a p .
- Cas recursiu: Si $m>1$, els divisors d' n menors que m són els divisors d' n menors que $m-1$, més $m-1$ si aquest és divisor d' n . HI.

Solució: divisors naturals d' n en una pila (IV)

Implementació de la funció recursiva i justificació:

```
void i_divisors(int n, stack<int> &p, int m)
/* Pre:  $n \geq m > 0$ ,  $p$  és buida */
/* Post:  $p$  conté els divisors d' $n$  menors que  $m$ ,
    en ordre decreixent des del cim */
{
    if (m > 1) {
        i_divisors(n, p, m-1);
        /* HI:  $p$  conté els divisors d' $n$  menors que  $m-1$ ,
            en ordre decreixent des del cim */
        if (n % (m-1) == 0) p.push(m-1);
    }
}
```

- Cas senzill: Si $m=1$, no hi ha cap divisor d' n menor que m . No afegim res a p .
- Cas recursiu: Si $m>1$, els divisors d' n menors que m són els divisors d' n menors que $m-1$, més $m-1$ si aquest és divisor d' n . **HI**.
- Acabament: m decreix a cada crida recursiva.

Índex

1 Immersió o generalització d'una funció

- Immersió per afebliment de la postcondició
- Immersió per reforçament de la precondició
- Transformació d'alg. recursius lineals finals a alg. iteratius

2 Exercicis

Immersió per reforçament de la precondició

- Per què?

¹P.e., recursivitat final amb postcondició constant.

Immersió per reforçament de la precondició

- Per què? Perquè la funció original no admet una solució recursiva o perquè es vol una solució alternativa¹.

¹P.e., recursivitat final amb postcondició constant.

Immersió per reforçament de la precondició

- Per què? Perquè la funció original no admet una solució recursiva o perquè es vol una solució alternativa¹.
- Com és la funció recursiva?

¹P.e., recursivitat final amb postcondició constant.

Immersió per reforçament de la preconditionió

- Per què? Perquè la funció original no admet una solució recursiva o perquè es vol una solució alternativa¹.
- Com és la funció recursiva?
 - Té una preconditionió reforçada amb una versió feble de la postcondició original. El que s'afegeix a la `Pre` ha de ser fàcil de complir a la primera crida.

¹P.e., recursivitat final amb postcondició constant.

Immersió per reforçament de la preconditionió

- Per què? Perquè la funció original no admet una solució recursiva o perquè es vol una solució alternativa¹.
- Com és la funció recursiva?
 - Té una preconditionió reforçada amb una versió feble de la postcondició original. El que s'afegeix a la `Pre` ha de ser fàcil de complir a la primera crida.
 - Té més paràmetres que la funció original.

¹P.e., recursivitat final amb postcondició constant.

Immersió per reforçament de la preconditionió

- Per què? Perquè la funció original no admet una solució recursiva o perquè es vol una solució alternativa¹.
- Com és la funció recursiva?
 - Té una preconditionió reforçada amb una versió feble de la postcondició original. El que s'afegeix a la `Pre` ha de ser fàcil de complir a la primera crida.
 - Té més paràmetres que la funció original.
 - Té la mateixa postcondició que la funció original.

¹P.e., recursivitat final amb postcondició constant.

Exemple 2: immersió per reforçament de la `Pre` (I)

Dissenyar una funció que donat un vector d'enters, retorni la suma de tots els seus elements.

Exemple 2: immersió per reforçament de la `Pre` (I)

Dissenyar una funció que donat un vector d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els
        elements del vector v */
```

Exemple 2: immersió per reforçament de la `Pre` (I)

Dissenyar una funció que donat un vector d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els
        elements del vector v */
```

- Fem immersió (afegim paràmetres) per trobar una solució recursiva. P.e., un paràmetre que faci “part” de la feina de la `Post`: afegim `suma_parcial` que contindrà la suma d'una part dels elements del vector.

Exemple 2: immersió per reforçament de la `Pre` (I)

Dissenyar una funció que donat un vector d'enters, retorni la suma de tots els seus elements. Especificació:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El resultat és la suma de tots els
        elements del vector v */
```

- Fem immersió (afegim paràmetres) per trobar una solució recursiva. P.e., un paràmetre que faci “part” de la feina de la `Post`: afegim `suma_parcial` que contindrà la suma d'una part dels elements del vector.
- Reforcem la `Pre` amb la informació del **nou paràmetre**.

Exemple 2: immersió per reforçament de la `Pre` (II)

- Proposta per la `Pre`:

Exemple 2: immersió per reforçament de la `Pre` (II)

- Proposta per la `Pre`:

```
/* Pre: v.size() > 0, 0 <= i <= v.size(),  
   suma_parcial = suma de v[0..i-1] */
```

Exemple 2: immersió per reforçament de la `Pre` (II)

- Proposta per la `Pre`:

```
/* Pre: v.size() > 0, 0 <= i <= v.size(),  
   suma_parcial = suma de v[0..i-1] */
```

- A la `Pre` hi apareix una variable, `i`, que passa a ser un **paràmetre extra** de la funció.

Exemple 2: immersió per reforçament de la `Pre` (II)

- Proposta per la `Pre`:

```
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
```

- A la `Pre` hi apareix una variable, `i`, que passa a ser un **paràmetre extra** de la funció.
- Especificació de la funció recursiva:

```
int ii_suma_vect_int(const vector<int> &v, int i,
                    int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els
   elements del vector v */
```

Exemple 2: immersió per reforçament de la Pre (III)

Implementació de la funció recursiva i justificació:

Exemple 2: immersió per reforçament de la Pre (III)

Implementació de la funció recursiva i justificació:

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

Exemple 2: immersió per reforçament de la `Pre` (III)

Implementació de la funció recursiva i justificació:

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

- Cas senzill: Si `i=v.size()`, `suma_parcial` conté la suma de tot el vector. Retornem el seu valor.

Exemple 2: immersió per reforçament de la Pre (III)

Implementació de la funció recursiva i justificació:

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

- Cas senzill: Si $i=v.size()$, `suma_parcial` conté la suma de tot el vector. Retornem el seu valor.
- Cas recursiu: Si $i \neq v.size()$, la crida recursiva ha d'acostar-nos al cas senzill. Si canviem i per $i+1$ caldrà que canviem `suma_parcial` per `suma_parcial+v[i]` per satisfer la Pre .

Exemple 2: immersió per reforçament de la `Pre` (III)

Implementació de la funció recursiva i justificació:

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

- Cas senzill: Si `i=v.size()`, `suma_parcial` conté la suma de tot el vector. Retornem el seu valor.
- Cas recursiu: Si `i!=v.size()`, la crida recursiva ha d'acostar-nos al cas senzill. Si canviem `i` per `i+1` caldrà que canviem `suma_parcial` per `suma_parcial+v[i]` per satisfer la `Pre`.
- Acabament: `v.size()-i` decreix a cada crida recursiva.

Exemple 2: immersió per reforçament de la Pre (IV)

Implementació de la funció original:

Exemple 2: immersió per reforçament de la `Pre` (IV)

Implementació de la funció original:

```
int suma_vect_int(const vector<int> &v)
/* Pre: v.size() > 0 */
/* Post: El valor retornat és la suma de
    tots els elements del vector v */
{
    return ii_suma_vect_int(v, 0, 0);
}
```

Comproveu que la `Pre` de `ii_suma_vect_int` se satisfà a la primera crida.

Exercici: divisors naturals d' n en una pila

Donat un enter n més gran que zero, s'han d'obtenir tots els seus divisors naturals menors que ell en una pila ordenada en ordre decreixent des del cim. Per exemple, si $n = 24$, la pila ha de ser:

12
8
6
4
3
2
1

Repetiu aquest exercici, que hem resolt per afebliment de la `Post`, per reforçament de la `Pre`.

Índex

1 Immersió o generalització d'una funció

- Immersió per afebliment de la postcondició
- Immersió per reforçament de la precondició
- Transformació d'alg. recursius lineals finals a alg. iteratius

2 Exercicis

Recursivitat lineal, múltiple i final

Algorisme amb recursivitat **lineal**:

Algorisme recursiu que a cada crida recursiva genera solament una (o cap) crida recursiva.

Recursivitat lineal, múltiple i final

Algorisme amb recursivitat **lineal**:

Algorisme recursiu que a cada crida recursiva genera solament una (o cap) crida recursiva.

Algorisme amb recursivitat **múltiple**:

Algorisme recursiu que en alguna crida recursiva pot generar més d'una crida recursiva.

Recursivitat lineal, múltiple i final

Algorisme amb recursivitat **lineal**:

Algorisme recursiu que a cada crida recursiva genera solament una (o cap) crida recursiva.

Algorisme amb recursivitat **múltiple**:

Algorisme recursiu que en alguna crida recursiva pot generar més d'una crida recursiva.

Funció recursiva **lineal** és **final** si:

- 1 La darrera instrucció que s'executa és la crida recursiva, i
- 2 el resultat de la funció és el resultat que s'ha obtingut de la crida recursiva, sense cap modificació.

Exemples: recursivitat lineal final i no final (I)

- Recursivitat **lineal no final**: la que hem obtingut per afebliment de la `Post`.

```
int i_suma_vect_int(const vector<int> &v, int i)
/* Pre: v.size() > 0, 0 <= i < v.size() */
/* Post: El resultat és la suma de v[0..i] */
{
    int suma;
    if (i == 0) suma = v[0];
    else {
        suma = i_suma_vect_int(v, i - 1);
        suma = suma + v[i]; "NO_FINAL"
    }
    return suma;
}
```


Exemples: recursivitat lineal final i no final (II)

- Recursivitat **lineal final** (*tail recursion*): la que hem obtingut per reforçament de la `Pre`.

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if(i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v,i+1,suma_parcial+v[i]);
    return suma;
}
```

Recursivitat lineal final amb `Post` constant

Funció amb **postcondició constant**:

Quan la postcondició no depèn dels paràmetres que varien d'una crida a una altra, és a dir, la postcondició de la crida recursiva és la mateixa que la postcondició de la funció.

Recursivitat lineal final amb `Post` constant

Funció amb **postcondició constant**:

Quan la postcondició no depèn dels paràmetres que varien d'una crida a una altra, és a dir, la postcondició de la crida recursiva és la mateixa que la postcondició de la funció.

- Utilitat?
 - La versió iterativa d'un algorisme recursiu final és més simple que la d'un recursiu no final.

Recursivitat lineal final amb `Post` constant

Funció amb **postcondició constant**:

Quan la postcondició no depèn dels paràmetres que varien d'una crida a una altra, és a dir, la postcondició de la crida recursiva és la mateixa que la postcondició de la funció.

- Utilitat?
 - La versió iterativa d'un algorisme recursiu final és més simple que la d'un recursiu no final.
 - Si a més de recursiu final, té postcondició constant, la seva transformació a iteratiu és directa.

Recursivitat lineal final amb `Post` constant

Funció amb **postcondició constant**:

Quan la postcondició no depèn dels paràmetres que varien d'una crida a una altra, és a dir, la postcondició de la crida recursiva és la mateixa que la postcondició de la funció.

- Utilitat?
 - La versió iterativa d'un algorisme recursiu final és més simple que la d'un recursiu no final.
 - Si a més de recursiu final, té postcondició constant, la seva transformació a iteratiu és directa.
- La funció `ii_suma_vect_int` és recursiva final. Té postcondició constant?

Transformació d'algorismes recursius lineals finals a algorismes iteratius

```
Tipus2 f(Tipus1 x)
/* Pre: Q(x), x = X */
/* Post: R(X, s) */
{
    Tipus2 s;

    if (c(x)) s = d(x);
    else {
        s = f(g(x));
    }
    return s;
}
```

Transformació d'algorismes recursius lineals finals a algorismes iteratius

```

Tipus2 f(Tipus1 x)
/* Pre: Q(x), x = X */
/* Post: R(X, s) */
{
    Tipus2 s;

    if (c(x)) s = d(x);
    else {
        s = f(g(x));
    }
    return s;
}

```

```

Tipus2 f_iter(Tipus1 x)
/* Pre: Q(x) */
/* Post: R(x, s) */
{
    Tipus2 s;

    while (not c(x)) x = g(x);

    s = d(x);

    return s;
}

```

Exemple: transformació de recursivitat final a iteratiu (I)

Partim de la versió recursiva final de l'exemple de sumar els elements d'un vector d'enters.

Exemple: transformació de recursivitat final a iteratiu (I)

Partim de la versió recursiva final de l'exemple de sumar els elements d'un vector d'enters.

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

Exemple: transformació de recursivitat final a iteratiu (I)

Partim de la versió recursiva final de l'exemple de sumar els elements d'un vector d'enters.

```
int ii_suma_vect_int(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;
    if (i==v.size()) suma = suma_parcial;
    else suma = ii_suma_vect_int(v, i+1, suma_parcial+v[i]);
    return suma;
}
```

- Observem que `ii_suma_vect_int` té tres paràmetres i n'hi ha dos que canvien a cada crida recursiva: `i`, `suma_parcial`.

Exemple: transformació de recursivitat final a iteratiu (II)

Amb la transformació descrita obtenim:

Exemple: transformació de recursivitat final a iteratiu (II)

Amb la transformació descrita obtenim:

```
int ii_suma_vect_int_iter(const vector<int> &v, int i, int suma_parcial)
/* Pre: v.size() > 0, 0 <= i <= v.size(),
   suma_parcial = suma de v[0..i-1] */
/* Post: El resultat és la suma de tots els elements de v */
{
    int suma;

    while(i != v.size()){
        suma_parcial += v[i];
        ++i;
    }
    suma = suma_parcial;

    return suma;
}
```

Índex

1 Immersió o generalització d'una funció

2 Exercicis

Exercicis per practicar la tècnica d'immersió

Els exercicis següents s'han de resoldre de forma **recursiva**:

- Exercici 1: Retornar l'element central (el del mig) d'una llista d'enters de longitud senar.
- Exercici 2: Buscar un punt amb la primera coordenada igual a un valor x donat en un vector de punts.
En cas de trobar-ne algun, la funció ha de retornar el booleà `true` i la primera posició en què l'ha trobat; altrament, ha de retornar el booleà `false` i com a posició el valor -1 .
- Exercici 3: Comprovar si dues llistes d'enters són iguals, és a dir, si contenen els mateixos elements en el mateix ordre.